

ok give me complexity of all operations on a minheap

Perfect, this *is* one of those interview-must-know tables. Let's keep it crisp and intuitive.

Below is **time complexity for all common operations on a Min-Heap** (binary heap, array-based).

---

## Min-Heap Operation Complexities

### Heap Properties (quick reminder)

- Complete binary tree
- Min element always at the root
- Height =  $O(\log n)$

---

### Core Operations

| Operation                | What happens                           | Time Complexity |
|--------------------------|----------------------------------------|-----------------|
| Peek / Get Min           | Read root element                      | $O(1)$          |
| Insert                   | Add at end → heapify up                | $O(\log n)$     |
| Extract Min              | Remove root → move last → heapify down | $O(\log n)$     |
| Decrease Key             | Decrease value → heapify up            | $O(\log n)$     |
| Increase Key             | Increase value → heapify down          | $O(\log n)$     |
| Delete (arbitrary index) | Swap with last → heapify               | $O(\log n)$     |

---

### Construction & Bulk Ops

| Operation                             | Description                       | Time Complexity                 |
|---------------------------------------|-----------------------------------|---------------------------------|
| <b>Build Heap (heapify)</b>           | Bottom-up construction from array | <b><math>O(n)</math></b>        |
| <b>Insert n elements (one-by-one)</b> | Repeated insert                   | <b><math>O(n \log n)</math></b> |
| <b>Heap Sort</b>                      | Build heap + n extractions        | <b><math>O(n \log n)</math></b> |

## Search & Traversal

| Operation                | Time                     |
|--------------------------|--------------------------|
| Search arbitrary element | <b><math>O(n)</math></b> |
| Traversal                | <b><math>O(n)</math></b> |

⚠️ Heap is **not ordered globally**, only parent  $\leq$  children  
 → no binary search possible

## Space Complexity

| Aspect          | Space                                  |
|-----------------|----------------------------------------|
| Heap storage    | <b><math>O(n)</math></b>               |
| Auxiliary space | <b><math>O(1)</math></b> (array-based) |

## Why These Complexities Make Sense (Interview Intuition)

- **$O(\log n)$**  ops = movement along tree height
- **Peek is  $O(1)$**  because min is always at index 0
- **Build heap is  $O(n)$**  because:
  - Most nodes are near leaves
  - Few nodes near root do expensive heapify

This  **$O(n)$  build heap** vs  **$O(n \log n)$  insert-all** distinction is *very* interview-relevant.

## Python heapq Mapping (quick sanity check)

| Python API                    | Complexity  |
|-------------------------------|-------------|
| <code>heapq.heappush()</code> | $O(\log n)$ |
| <code>heapq.heappop()</code>  | $O(\log n)$ |
| <code>heap[0]</code>          | $O(1)$      |
| <code>heapq.heapify()</code>  | $O(n)$      |

If you want, next we can:

- compare **min-heap vs max-heap** (trick questions)
- do **top-k** complexity tradeoffs
- or walk through **why heapify is  $O(n)$**  step-by-step (whiteboard style)

Just say the word 🗨️



what are decrease key, you mean changing the value of the root node?

Great catch — and **no**, it does **not** mean changing only the root. This is a *very common confusion*, so let's clean it up properly.

## What is Decrease-Key in a Min-Heap?

**Decrease-key** means:

Lowering the value of *any specific node* in the heap,  
then restoring the min-heap property.

📌 It is **not limited to the root**.

---

## Precise Definition

Given:

- a min-heap
- an index `i`
- a new value `new_val` such that  
`new_val < heap[i]`

**Decrease-key(*i*, *new\_val*):**

1. Set `heap[i] = new_val`
  2. **Heapify UP** (bubble up) until heap property holds
- 

## Why Heapify Up?

Because:

- You **made the node smaller**
  - It might now be **smaller than its parent**
  - So it can only violate the heap property **upward**
- 

## Example

Initial min-heap (array representation):

makefile

📄 Copy code

Index: 0 1 2 3 4 5  
Heap: [1, 4, 3, 10, 8, 5]

Now do:

scss

📄 Copy code

```
decrease_key(index=3, new_val=2)
```

Step 1 — change value:

```
csharp
```

[Copy code](#)

```
[1, 4, 3, 2, 8, 5]
```

Step 2 — heapify up:

- $2 < 4 \rightarrow \text{swap}$

Final heap:

```
csharp
```

[Copy code](#)

```
[1, 2, 3, 4, 8, 5]
```

---

## Is the Root a Special Case?

Yes — but trivial.

If you **decrease the root**:

- It has no parent
- Heap property is already satisfied
- **Time =  $O(1)$**

But **decrease-key is defined for arbitrary nodes**, not just the root.

---

## Why Does This Operation Matter?

### Graph Algorithms

- **Dijkstra's shortest path**
- **Prim's MST**

They constantly:

- find a node

- **decrease its key**
- re-order the heap

Without efficient decrease-key → algorithms get slower.

---

## Complexity (Min-Heap)

| Operation         | Time        |
|-------------------|-------------|
| Decrease-key      | $O(\log n)$ |
| Root decrease-key | $O(1)$      |

---

## Python `heapq` Caveat (Very Interview-Relevant ⚠)

Python's `heapq` :

- **✗ does NOT support decrease-key directly**
- No index tracking

Workarounds:

1. Push `(new_value, node)` again
2. Mark old entries as invalid (lazy deletion)
3. Use a custom heap with index map

Interviewers **love** asking this follow-up.

---

## One-line takeaway (say this in interviews)

“Decrease-key lowers the value of **any heap element**, then restores order by bubbling it **up**, not necessarily the root.”

If you want, I can:

- show **delete-key implemented using decrease-key**
- connect it directly to **Dijkstra step-by-step**
- or contrast it with **increase-key** (symmetric but heapify down)

Just tell me 🐼

